

PIPEWEAVE: Synergizing Analytical and Learning Models for Unified GPU Performance Prediction

Kaixuan Zhang^{1,3}, Yunfan Cui¹, Shuhao Zhang¹, Chutong Ding¹, Shiyu Qian^{1,*}, Luping Wang^{2,*}, Jian Cao¹, Guangtao Xue¹, Cheng Huang², Guodong Yang², and Liping Zhang²

¹Shanghai Jiao Tong University, China ²Alibaba Group, China *Corresponding authors

³Work done during an internship at Alibaba Group
{zks1anx, cuiyunfan, zhang-shuhao, qshiyu}@sjtu.edu.cn

Abstract—The rapid expansion of Transformer-based large language models has dramatically increased the need for high-performance GPUs. As a result, there is growing demand for fast, accurate, and widely generalizable GPU performance models to support next-generation hardware selection and system-level exploration. However, current data-driven methods are limited, exhibiting poor generalization across hardware and inadequate modeling of complex production-level kernels common in modern inference stacks. To address these issues, we present PIPEWEAVE, a unified GPU modeling framework. This approach first employs an analytical model to quantify a given kernel’s demands on the GPU’s heterogeneous instruction pipelines. These analytical features are then fed into a machine learning (ML) model to capture complex cross-pipeline interactions and resource dependencies, enabling high-fidelity performance prediction. Our evaluation across 11 GPU types from four generations of major architectures on two widely-used serving systems demonstrates that PIPEWEAVE delivers high fidelity and strong generalizability. It achieves accurate predictions, with only 6.1% average error at the kernel level and 8.5% for end-to-end inference—reducing the error of state-of-the-art methods by 6.7× and 4.4×, respectively. We also demonstrate PIPEWEAVE’s value “beyond simulation” by utilizing its performance ceiling to diagnose implementation shortcomings and guide the optimization of a production fused MoE Triton kernel, achieving up to 1.7× speedup. Code is available online.¹

I. INTRODUCTION

The advent of Transformer-based [70] large language models (LLMs) has fundamentally reshaped the landscape of artificial intelligence. Today, a vast array of LLMs—ranging from proprietary flagships like Gemini [16] to open-source series like Llama [67] and Qwen [3]—are deployed to power diverse services, from coding assistants to document summarization. Efficiently serving these varied workloads across heterogeneous hardware platforms requires large-scale clusters of high-performance nodes.

The surging demand for performance is met by a relentless pace of hardware innovation from major vendors like NVIDIA and AMD, who frequently release new GPU architectures with substantial performance and feature updates. For example, since the introduction of its Ampere architecture, NVIDIA has released **four** distinct architectures and **dozens** of non-consumer model variations targeting different market segments [39]. This rapid co-evolution of models and hardware

poses a critical challenge for system designers and architects. The sheer volume of hardware configurations makes exhaustive testing impractical, while the inability to acquire every possible configuration—or even access unreleased, next-generation hardware—further compounds this issue. Therefore, to facilitate large-scale system-design, hardware selection, and the development of next-generation systems, the need for fast, accurate, and generalizable GPU performance models has never been more pressing.

Historically, GPU performance modeling has followed three main paradigms, each exhibiting a different trade-off among fidelity, speed, and generality. First, cycle-accurate simulators [4], [22], [66] offer the highest fidelity by emulating microarchitectural behavior in detail, but their simulation speed is computationally expensive and their lack of portability makes generalizing to new or undocumented hardware challenging. Second, analytical models [6], [19], [25] provide much faster estimates by relying on performance formulas such as interval analysis, yet their accuracy is often constrained and their dependence on hardware-specific microbenchmarking restricts generality to unseen architectures. Third, data-driven approaches [26], [76] achieve high speed by learning tile-level latency from measurements, but their predictive accuracy can be variable and their high-level modeling assumptions—treating tiles as atomic, assuming uniform SM behavior, and not fully capturing fused-kernel coupling (e.g., FlashAttention [11])—can impact generalization across workloads and hardware generations.

To bridge this gap, we present PIPEWEAVE, a framework that achieves high fidelity, fast speed, and broad generalizability in GPU performance modeling through a combined analytical–ML design that *weaves* pipeline-level analysis into accurate predictions. The framework first decomposes a given kernel into a set of fundamental *tasks*, each representing a schedulable unit of work for a Streaming Multiprocessor (SM). It then simulates how these tasks are mapped onto SMs according to the kernel’s execution paradigm, producing a realistic task distribution. Based on this distribution, PIPEWEAVE analytically derives each task’s *pipeline demand* and associated *theoretical cycles* for the SM’s heterogeneous instruction pipelines, and aggregates them into a compact multi-level feature set. Finally, a lightweight MLP consumes

¹<https://github.com/zksainx/pipeweave>

TABLE I
RUNTIME BREAKDOWN OF QWEN2.5-32B INFERENCE ON A 4×A100
CLUSTER WITH TENSOR PARALLELISM (TP=4).

Phase	GEMM	Attention	RMSNorm	SiLU&Mul	All-Reduce	Other
Prefill	72.70%	8.22%	3.85%	2.26%	12.10%	0.87%
Decode	65.05%	17.78%	3.19%	1.50%	5.76%	6.72%

these features to predict the kernel’s execution duration.

We conducted extensive evaluations to validate our framework. Our experimental testbed spans 4 hardware generations, encompassing 11 distinct GPU types (6 for training, 5 for unseen testing), and 5 categories of critical kernels (e.g., GEMM, Attention) in FP8, BF16/FP16, and FP32 precisions, commonly invoked by frameworks like vLLM [72] and SGLang [59]. At the kernel level, PIPEWEAVE achieves a low average MAPE of 6.1% on seen GPUs and 11.4% on unseen GPUs, drastically outperforming the state-of-the-art (SOTA) baseline, Neusight [26], representing an error reduction of $6.7\times$ and $3.8\times$, respectively. We further validate our model on complex inference workloads, using three large models (Qwen2.5-14B, Qwen3-32B, Llama3.1-70B) with various Tensor and Pipeline Parallel (TP/PP) configurations. In these E2E scenarios, PIPEWEAVE maintains high fidelity, achieving an average error of 8.5% on seen GPUs and 10.7% on unseen GPUs—reducing the prediction error of Neusight by $4.4\times$ and $3.1\times$. Finally, we demonstrate PIPEWEAVE’s value “beyond simulation.” By utilizing the model to establish a potential performance ceiling, we identify hardware-specific implementation inefficiencies in a production Fused MoE Triton kernel and guide targeted optimizations, achieving up to a $1.7\times$ speedup.

In summary, this paper makes the following contributions.

- **Unified Modeling Framework:** We propose PipeWeave, a unified framework synergizing analytical modeling with machine learning to accurately capture complex pipeline interactions for high-fidelity prediction.
- **Superior Generalization:** Validated across 11 GPUs spanning four generations, PIPEWEAVE achieves SOTA accuracy on unseen architectures, reducing prediction error by up to $6.7\times$ over prior methods.
- **Optimization Guidance:** We demonstrate utility “beyond simulation” by establishing performance ceilings to diagnose implementation inefficiencies and guide targeted optimization for production kernels.

II. BACKGROUND

A. Large Language Models (LLMs)

Modern LLMs are predominantly based on the Transformer architecture [70]. A typical Transformer block includes two key components: a multi-head attention mechanism and a position-wise feed-forward network (FFN). LLM inference is commonly divided into two phases: *prefill* and *decode* [2], [59], [72]. During prefill, the input prompt is processed in parallel, with key/value pairs computed and stored in the KV cache for every token. The decode phase then generates output tokens in an autoregressive way.

Table I shows the kernel (a GPU-executable function) runtime distribution for Qwen2.5-32B inference on a 4×A100 cluster with Tensor Parallelism (TP=4) (batch size 8, sequence length 8192). These categories correspond to core computational building components and communication primitives of the Transformer architecture used in today’s mainstream distributed LLMs: GEMM kernels [2] dominate the workload, stemming from linear projections in both attention and feed-forward layers; Attention kernels compute the relationships prior to attention and feed-forward computations; operations like SiLU&Mul implement activation functions and element-wise calculations, which are central to the SwiGLU FFN used in many LLMs [61]; and All-Reduce kernels handle the essential collective communications across GPUs. Together, these major kernels account for the vast majority of the total runtime. As their dominance persists across models, software stacks, and hardware generations [2], [11], [64], our analysis focuses on accurately modeling these kernels.

Furthermore, recent advances in LLM optimization have led to the adoption of specialized kernels. The use of lower-precision data types, notably FP8 quantization for W8A8 (8-bit weights and activations) inference [24], [63], has popularized Scaled Matrix Multiplication (Scaled MM) kernels [40], [41], [45]. Concurrently, Mixture-of-Experts (MoE) architectures [9], [13], [62] leverage Fused MoE kernels. These kernels efficiently execute batched GEMM operations across expert sub-networks once token routing is finished.

B. From Kernels to GPU Architectures

Without loss of generality, our discussion is contextualized within the NVIDIA GPU architecture, primarily focusing on LLM inference scenarios. To analyze performance in this context, we distinguish two tightly related perspectives: the *software* view, which involves how kernels are defined and launched, and the *hardware* view, which describes how kernels are mapped to and executed on the underlying microarchitecture, as illustrated in Figure 1.

Kernel execution involves two conceptual stages. The **compilation stage** produces GPU-executable code in the form of SASS instructions, which are the only representation that NVIDIA SMs can natively execute. In practice, the compilation toolchain may emit a fat binary that contains both architecture-specific SASS [33] and a virtual instruction set architecture (ISA) (PTX [54]); the CUDA driver may further JIT-compile PTX into SASS when necessary. Regardless of whether a kernel originates from Triton, CUDA C++, or precompiled libraries such as cuBLAS, its final execution always resolves to SASS instructions compatible with the target GPU microarchitecture [48], [68].

The **runtime stage** is responsible for launching the compiled kernel. A kernel launch specifies the CUDA execution configuration (Grid and its Cooperative Thread Arrays) [49]. The GPU hardware work-distribution logic schedules CTAs onto SMs, dynamically dispatching work in units of CTAs rather than mapping the entire Grid at once [34], [47]. At

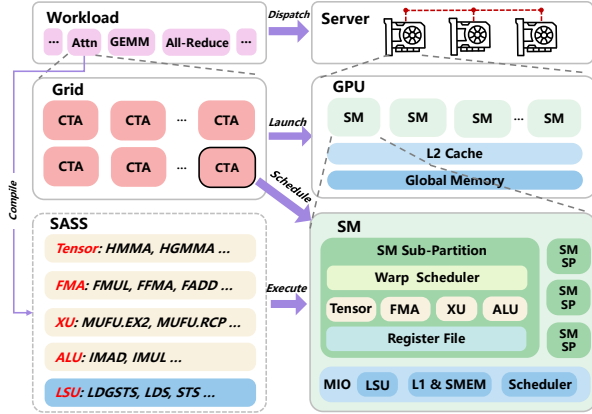


Fig. 1. An illustration of the mapping between the software hierarchy and the physical GPU hardware hierarchy.

the SM level, the SASS instructions from compilation are fetched via the instruction cache hierarchy and issued by warp schedulers to the SM's diverse execution pipelines.

An SM in Ampere [36] and later architectures is typically partitioned into four SM Sub-partitions (SMSPs) and one Memory I/O (MIO) unit. Each SMSP mainly includes a Warp Scheduler for cycle-by-cycle instruction dispatch, a Register File, and a collection of specialized math pipelines. Notable examples are fixed-latency math pipelines such as FMA (processing most FP32 arithmetic operations like FMUL, FADD) and Tensor (executing MMA instructions like HMMA), as well as variable-latency math pipelines like the XU (for special functions such as base-2 exponential MUFU.EX2). The MIO unit is dedicated to managing data movement. It incorporates on-chip memory caches—the L1 cache and Shared Memory (SMEM)—as well as the Load Store Unit (LSU), which executes memory instructions (e.g., LDGSTS, STS) for accessing global, local, or shared memory [52], [53].

This consistent high-level SM organization across Ampere and later architectures provides a stable micro-architectural abstraction. PIPEWEAVE builds on this foundation to achieve generalizability across kernels and hardware platforms.

III. MOTIVATION

The rapid co-evolution of LLMs and GPUs necessitates performance modeling tools that are fast, accurate, and generalizable across diverse LLMs, serving frameworks, and hardware architectures. Current approaches, such as cycle-accurate simulation [4], [22], analytical modeling [6], [19], [25], and data-driven methods [1], [73], each offer partial solutions, yet none fully meet these requirements. Although cycle-accurate simulators deliver high fidelity, they are slow. Analytical models are labor-intensive and lack generalizability.

In this landscape, data-driven strategies—particularly “grey-box” approaches that integrate analytical modeling with ML techniques [26], [76]—represent a distinct paradigm. These methods aim to deliver rapid predictions with improved generalizability by utilizing strategies like tile-level decomposition and incorporating hardware specifications. However, despite

marking notable progress, even these advanced techniques can encounter modeling constraints that limit their accuracy, with prediction errors exceeding 40% (Section VI-C).

The core issue arises from insufficient microarchitectural fidelity. Specifically, while state-of-the-art simulators like Neusight [26] incorporate workload-level feature engineering (e.g., tile decomposition), they still fall short of true micro-architectural modeling in three key aspects.

Mismatched Granularity: Although such methods decompose workloads into thread block tiles, their hardware representation remains coarse-grained. They primarily rely on tile-level descriptors as inputs to an ML model, without explicitly modeling how a tile's execution translates into specific demands and contention on the SM's heterogeneous instruction pipelines. For instance, the execution of a single tile inherently involves concurrent activity across multiple hardware units—such as Tensor, FMA, and memory pipelines. A tile-centric abstraction therefore collapses heterogeneous pipeline activity into a single aggregate workload representation, effectively treating the SM as a monolithic black box rather than reflecting its actual execution behavior.

Inability to Model Fused Kernels: Such methods model performance primarily at the level of individual deep learning operators in the computation graph (e.g., GEMM, Softmax). This abstraction assumes that kernel execution can be approximated as a composition of standard operators. However, modern high-performance implementations increasingly rely on fused kernels (e.g., FlashAttention [11]), where multiple operators are tightly integrated into a single GPU kernel. In such kernels, performance is governed by tightly coupled execution patterns introduced by operator fusion, where multiple computations are executed within a shared execution structure and intermediate data is reused across steps. As a result, the effective computation and data movement behavior no longer aligns with the boundaries of individual operators. These execution characteristics cannot be accurately captured through operator-level decomposition, leading to significant modeling limitations.

Static Wave Modeling: While current baselines attempt to account for wave quantization (the tail effect of thread block scheduling), they typically rely on a static assumption that tiles within a wave exhibit uniform execution latency. In practice, dynamic workloads—such as causal attention processing variable-length tokens or kernels with early-exit conditions—introduce substantial tile-to-tile latency variation. Without a granular scheduling model, these approaches struggle to capture cross-SM load imbalance and tail effects that frequently degrade real-world performance.

These limitations highlight the need for a new modeling approach that more faithfully reflects GPU execution. An accurate and generalizable model must bridge high-level workload structure with the microarchitectural realities of how kernels execute on modern GPUs. In particular, it should explicitly characterize how workload decomposition and scheduling translate into concrete demands on heterogeneous instruction pipelines, rather than treating execution as an abstract work-

load description. At the same time, purely analytical modeling is insufficient to capture the complex interactions and resource coupling that arise in real kernels, and such models are often tailored to specific operators and hardware assumptions, making them difficult to generalize as new kernels or GPU architectures emerge without significant re-derivation. Therefore, a hybrid design is needed: one that grounds the model in execution-aware analytical structure while leveraging learning-based components to capture higher-order performance effects. This design philosophy underpins PIPEWEAVE.

IV. THE PIPEWEAVE DESIGN

Achieving accurate and generalizable GPU performance prediction requires a comprehensive understanding of the intricate interplay between software kernels and underlying hardware architectures. A robust modeling approach must account for both deterministic first-order effects and complex dynamic interactions. Accordingly, we propose PIPEWEAVE, a framework built on a methodology guided by the *dual principles of knowledge and data*.

The *knowledge-driven* component is a hierarchical analytical model that leverages deep domain-specific knowledge of the GPU’s parallel execution model to systematically decompose a kernel’s complex execution flow. This top-down decomposition progresses from the entire kernel to a set of fundamental tasks, and further into the elemental demands on specific instruction pipelines. This decomposition yields an interpretable feature set for the complementary *data-driven* component: a lightweight MLP designed to capture the complex non-linear interactions and resource contention, which are challenging to characterize analytically. It is this integration of knowledge-driven decomposition and data-driven modeling for higher-order effects that enables PIPEWEAVE to achieve high-fidelity performance predictions.

PIPEWEAVE comprises four core modules, as shown in Figure 2: (1) **Kernel Decomposer**, which breaks down a kernel’s overall execution into a set of fundamental tasks (§IV-A); (2) **Scheduling Simulator**, modeling how tasks are assigned to the GPU’s SMs and producing the final task distribution (§IV-B); (3) **Feature Analyzer**, converting the task distribution into a multi-level feature set that captures instruction pipeline demands and associated theoretical cycles (§IV-C); and (4) **Performance Estimator**, which synthesizes these features into a final prediction using a lightweight MLP to model complex higher-order interactions (§IV-D).

This multistage design underpins PIPEWEAVE’s generalizability. The initial two modules ensure *kernel generalizability* by converting any kernel into a uniform task distribution, agnostic to its source. The third module then enables *hardware generalizability* by mapping this distribution to a feature set via a compact vector representing the target GPU’s architectural parameters. Once the MLP for a given kernel is trained across various hardware configurations, predicting performance for any new input or GPU—even unseen architectures—becomes highly efficient. The process only involves running fast analytical steps to produce the corresponding feature vector, then

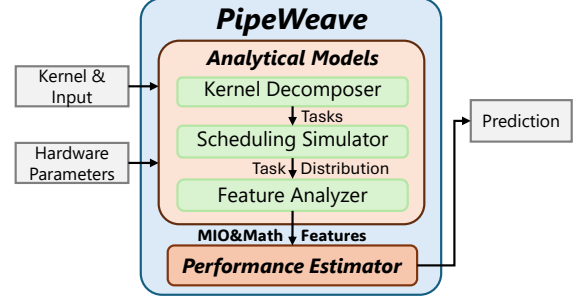


Fig. 2. Overview of the PIPEWEAVE modeling framework, detailing the flow from kernel decomposition to the final performance prediction.

performing one forward pass of the MLP, enabling real-time predictions.

While our evaluation is validated on NVIDIA GPU architectures (Table VI), the principle of decomposing a kernel into its demands on heterogeneous instruction pipelines is fundamentally general. This can be readily extended to other modern accelerators, such as AMD GPUs.

A. Kernel Decomposer

To accurately capture the parallel execution of modern GPUs as described in Section II-B, PIPEWEAVE decomposes a kernel’s workload into a set of smaller *tasks*. This decomposition is central to our approach, as it models the kernel in a manner consistent with GPU parallelism [47]. Although prior studies [26], [76] have explored partitioning kernels into tiles, they often rely on inferring simplified tiling logic from profiling data. In contrast, PIPEWEAVE emphasizes deterministic analysis of available source code. This enables a more accurate and verifiable decomposition process, capturing complex and diverse task structures in modern kernels.

The precise definition of a task can vary across GPU architectures and kernel implementations. In the *conventional GPU execution model* [47] (e.g., FlashAttention-2 [10]), a task usually corresponds to a Cooperative Thread Array (CTA), also known as a thread block. A kernel launch generates a grid of CTAs, and the hardware scheduler assigns each CTA to one available SM for the duration of its execution. However, in modern high-performance GPU paradigms such as *persistent kernels* used in patterns like Ping-Pong GEMM [27], [50], this one-to-one mapping no longer holds. Under this execution model, a long-lived CTA stays resident on an SM and serves as a persistent worker. Therefore, the fundamental schedulable unit—our *task*—is not the CTA itself, but a smaller computational packet that the resident CTA fetches from a global work queue.

While the fundamental decomposition methodology is consistent, the specific implementation varies by kernel. To characterize a task’s execution properties, our framework identifies *dimensional parameters* (d_i) that define its scope and scale. While these dimensional parameters, and hence the computational workload, are often uniform across all tasks in a kernel (e.g., each GEMM task is typically defined by the same tile dimensions ($tile_M, tile_N, tile_K$)), this is not always the

TABLE II
HARDWARE SPECIFICATIONS REQUIRED BY PIPEWEAVE.

Parameter	Value Range	Unit
Compute Capability	8.0 – 12.0	-
Number of SMs	78 – 188	-
SM Clock Frequency	1410 – 2520	MHz
Tensor Pipe Throughput	512 – 4096	ops/cycle/SM
FMA Pipe Throughput	64 – 128	ops/cycle/SM
XU Pipe Throughput	16	ops/cycle/SM
Global Memory Bandwidth	696 – 4916	GB/s
L2 Cache Bandwidth	2430 – 10400	GB/s
Shared Memory Bandwidth per SM	128	Byte/cycle
Shared Memory Size per SM	100 – 228	KB
Register File Size per SM	256	KB

case. A key exception occurs in FlashAttention [10], [11], [60], [75] when causal masking is applied. Due to the causal constraint, tasks processing earlier query tokens attend to fewer key/value tokens than those handling later tokens. Thus, even if the nominal task dimensions seem uniform, the actual workload per task can differ significantly.

We formalize the process of deriving these tasks and their parameters through a mapping function $\mathcal{F}$. For a given kernel, $\mathcal{F}$ maps the input parameters $\mathbf{X}$ and the hardware’s architectural specifications $\mathbf{S}$ (Table II) to the full set of tasks $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_t\}$:

$$\{\tau_1, \tau_2, \dots, \tau_t\} = \mathcal{F}(\mathbf{X}, \mathbf{S}) \quad (1)$$

Each task τ_i encapsulates a specific part of the kernel’s workload, characterized by its dimensional parameter vector $\mathbf{d}_i$. These parameters form the basis for analytically deriving the task’s execution properties, such as computational and memory demands, as detailed in the subsequent section (§IV-C).

The method for deriving the decomposition function $\mathcal{F}$ depends on kernel accessibility. For open-source libraries (e.g., FlashInfer [75]), $\mathcal{F}$ is derived by directly extracting the parallelization strategy and thread block mapping logic from the source code. However, this approach does not apply to closed-source libraries such as NVIDIA’s cuBLAS [51]. To handle such case, we infer the mapping function empirically. For example, to identify the decomposition logic for a cuBLAS GEMM kernel running in BF16 precision, we profile its execution over diverse input matrix dimensions (M, N, K) using tools like the PyTorch Profiler [56]. By analyzing the profiled data, particularly the correlation between kernel names, the number of CTAs, and input sizes, we reverse-engineer the kernel’s implicit task partitioning strategy. This empirical approach enables us to build a surrogate mapping function $\mathcal{F}$ that closely approximates the proprietary decomposition logic.

B. Scheduling Simulator

A kernel’s performance is determined not only by its total workload but also by how that work is allocated across the GPU’s parallel resources. After decomposing the kernel into an abstract set of tasks, the next key component of our framework is to simulate the scheduling of these tasks onto SMs. This scheduling analysis converts the task set into a concrete *task*

distribution, providing a precise mapping of tasks to specific SMs. This mapping is crucial, as it enables accurate per-SM characterization of the kernel’s behavior and helps identify performance bottlenecks resulting from workload imbalance—a critical aspect overlooked in prior studies [26], [29], [74], [76]. They often rely exclusively on aggregated kernel-level metrics and assume an over-simplified scheduling model where all tasks are handled uniformly. PIPEWEAVE is designed for versatility, supporting the two main scheduling paradigms used in modern GPU applications.

Hardware-Implemented Scheduler. For conventional kernels, task scheduling is handled by the GPU’s hardware scheduler, called the *GigaThread Engine* [28], [47]. Since the exact behavior of this hardware component is not publicly documented, its default scheduling policy is generally inferred from empirical studies to be round-robin (RR) [18], [20], [21], [28], [30], [31], [35], [65], [79]. The policy first assigns each SM at least one task (i.e., a CTA). If an SM still has enough resources (e.g., registers, shared memory, warp-slots, etc.) to support additional tasks, a second assignment round is performed. This rounding-assignment process continues until all SMs are saturated, either due to resource constraints or hardware limits. Afterwards, a new task is assigned to an SM when an existing task finishes and retires from it.

Software-Implemented Scheduler. For persistent kernels, the role of hardware scheduler in dispatching CTAs becomes secondary, as each CTA launches only once and remains resident on an SM during execution. Key scheduling logic is handled in software. In this setup, a long-lived CTA repeatedly processes fine-grained work units taken from a global list. In GEMM-like kernels, these units are commonly implemented as *tiles*, which represent the concrete form of our *tasks*. Tile assignment is managed by a tile scheduler [50], [71], a software component with logic specific to the kernel.

By simulating these scheduling mechanisms, PIPEWEAVE accurately derives a realistic *task distribution*. We formalize the distribution as a partition of the total task set, $\mathcal{T} = \{\tau_1, \tau_2, \dots, \tau_t\}$, across available SMs. This partition comprises sets, $\{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_{N_{SM}}\}$, where N_{SM} denotes the SM count and each set $\mathcal{T}_j$ contains all tasks assigned to the j -th SM. Our scheduling simulator, represented by mapping function $\mathcal{M}$, generates this partition as follows:

$$\{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_{N_{SM}}\} = \mathcal{M}(\mathcal{T}, \mathbf{S}) \quad (2)$$

The sets $\{\mathcal{T}_j\}$ form a partition of $\mathcal{T}$, such that $\bigcup_{j=1}^{N_{SM}} \mathcal{T}_j = \mathcal{T}$ and $\mathcal{T}_i \cap \mathcal{T}_j = \emptyset$ for $i \neq j$.

C. Feature Analyzer

Feature engineering is conceptually guided by principles from the Roofline performance model [74]. This classic model offers a powerful first-order analysis by determining whether a kernel is bound by the hardware’s peak compute throughput or memory bandwidth. However, its predictive accuracy for modern GPUs remains limited. This occurs because its high-level, two-dimensional view of compute and memory fails to

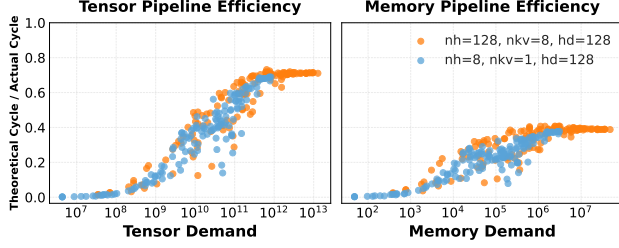


Fig. 3. Illustration of the PIPEWEAVE multi-dimensional analysis for FlashAttention-2 on A100. As demand increases, measured performance for two different configurations approaches the theoretical “roof” and plateaus.

capture the intricate resource contention and dynamic interactions that arise when complex modern kernels execute on heterogeneous hardware.

To overcome this limitation, PIPEWEAVE expands the Roofline model into a *multi-dimensional analysis*. Instead of a single compute roof and a single memory roof, our model calculates a separate theoretical performance limit for every key instruction pipeline. This necessitates characterizing kernel execution along two fundamental dimensions: (1) **Demand**, measuring the total workload (e.g., operations or bytes) applied to each pipeline; (2) **Theoretical Cycles**, obtained from the demand, indicating the ideal execution time if that pipeline alone were the bottleneck. This resembles a particular pipeline’s “roof”. Figure 3 shows a concrete example. It plots execution efficiency—the ratio of theoretical cycles to measured latency—against absolute pipeline demand. Unlike the standard roofline, pipelines are decoupled into separate plots, each showing a predictable and independent saturation trend.

Moreover, we do not construct rigid analytical models for complex instruction-level concurrency (e.g., the parallel execution of Tensor and FMA pipelines) or architecture-specific mechanisms (e.g., Hopper’s Tensor Memory Accelerator (TMA)). Accurately modeling such microarchitectural details would require generation-specific reverse engineering, which undermines cross-generation generalizability and significantly increases modeling complexity. Instead, PIPEWEAVE adopts a deliberate abstraction strategy. We unify diverse memory access mechanisms—ranging from conventional LSU instructions to advanced asynchronous copies—into generalized memory pipeline demands. By exposing these fundamental pipeline demands as separate raw features, we allow the model to learn their complex and non-linear interactions automatically in the subsequent MLP stage. Empirically, we find that this abstraction is sufficient to capture the dominant performance behaviors across architectures while maintaining strong generalizability.

The generation of these features follows a bottom-up process across three levels. First, at the *task* level, we characterize the isolated demands of both Math pipelines and MIO pipelines, deriving their corresponding per-task theoretical cycles. Next, these per-task features are aggregated to the *SM* level, creating a detailed profile for each SM and enabling identification of traits for the most heavily utilized SM. Finally,

TABLE III
PRIMARY OPERATIONS EXECUTED BY KEY MATH PIPELINES.

Math Pipeline	Primary Operations
Tensor	MMA instructions across various precisions (e.g., FP8, FP16, BF16).
FMA	FP32 floating-point add, multiply, and fused multiply-add.
XU	FP32 approximate floating-point special functions (e.g., reciprocal, reciprocal square root, base-2 logarithm, base 2 exponential, sine, cosine).

a second aggregation yields a whole-GPU profile containing demand and theoretical cycle metrics for all major pipelines.

1) *Math Pipelines*: For each task $\tau_i \in \mathcal{T}$, we define its computational demand per math pipeline by the number of executed operations it executes. These pipelines mainly process two operation types: matrix-multiply-accumulate (MMA) operations executed on the Tensor pipeline, and element-wise (EW) operations handled by units like FMA or XU pipelines. Key operations for each math pipeline [46], [47], [52], [53] are outlined in Table III.

For MMA operations in τ_i , the operation count ($N_{\text{ops, Tensor}}$) is derived directly from the task dimension vector $\mathbf{d}_i$, which includes the tile geometry $\{tile_M, tile_N, tile_K\}$. The total operation count is:

$$N_{\text{ops, Tensor}} = \alpha \cdot tile_M \cdot tile_N \cdot tile_K \quad (3)$$

Here, coefficient α represents the total number of basic multiply-add operations per output element during MMA computations. In a standard GEMM kernel [42], [43], one matrix multiplication gives $\alpha = 2$, while a FlashAttention kernel does two sequential matrix multiplications per task [11], resulting in $\alpha = 4$.

For the EW operations in task τ_i , our analysis directly computes the total operations (e.g., $N_{\text{ops, FMA}}$, $N_{\text{ops, XU}}$) for each math pipeline. This entails deriving the aggregate operation counts for specific hardware pipelines (Table III) by analyzing the kernel’s arithmetic expressions and loop iteration spaces.

Finally, for each pipeline p , the theoretical cycles C_p needed to execute these operations are determined by dividing the total operation count $N_{\text{ops, } p}$ by its corresponding throughput Th_p , a parameter from hardware specification $\mathbf{S}$:

$$C_p = \frac{N_{\text{ops, } p}}{Th_p} \quad (4)$$

After obtaining per-task demand features, we use a bottom-up approach to aggregate task distributions $\{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_{N_{SM}}\}$ into SM-level and GPU-level features. Starting at the SM level, for each pipeline p , we combine the demands of all tasks assigned to SM_{*j*} to compute total per-SM operations $N_{\text{ops, } p}^{\text{SM}_j}$ and theoretical cycles $C_p^{\text{SM}_j}$. These per-SM values are summed to obtain overall GPU operations $N_{\text{ops, } p}^{\text{GPU}}$. Corresponding GPU-level theoretical cycles are derived from this total workload and the combined throughput of pipeline p :

$$C_p^{\text{GPU}} = \frac{N_{\text{ops, } p}^{\text{GPU}}}{N_{SM} \cdot Th_p} \quad (5)$$

TABLE IV
THE ANALYTICAL FEATURE VECTOR PROVIDED AS INPUT TO THE MLP.

Pipeline	Granularity	Features
Math	GPU	Total Operations Total Theoretical Cycles
	SM	Max SM Operations Max SM Theoretical Cycles
MIO	GPU	Total Memory Demand Theoretical Cycles (Global, L2)
	SM	Max SM Memory Demand Theoretical Cycles (Global, L2, Shared)

2) *MIO pipelines*: For MIO pipelines, we measure total demand in bytes at three levels. First, for each task τ_i , we calculate the total *per-task* memory demand B_i by summing all data it loads from the memory hierarchy. This approach is taken because loads are often on the critical execute path in most kernels. A data stall directly affects consumer latency (math pipelines) [53]. Using the task distribution, these per-task values are summed for tasks in set $\mathcal{T}_j$ to get the *per-SM* memory demand B^{SM_j} . Finally, summing all per-SM values B^{SM_j} gives the *global* memory demand B^{GPU} .

From these aggregated byte counts, we derive several theoretical cycle features. The theoretical cycles C_{mem} is calculated by dividing total bytes at a given level by a specific memory subsystem’s theoretical bandwidth, expressed as $C_{mem} = B/BW_{mem}$. At GPU-level, we apply this formula with B^{GPU} , using L2 Cache and Global Memory bandwidths. At SM-level, B^{SM_j} is used along with per-SM bandwidths for Shared Memory, L2 Cache, and Global Memory.

D. Performance Estimator

The final component of PIPEWEAVE is a lightweight machine learning model that predicts the overall kernel execution duration. The MLP uses a single feature vector as input, which is the concatenation of all analytical features from earlier stages (Section IV-C). This vector includes features (Table IV) from the MIO pipeline, plus features from one or more Math pipelines based on the kernel’s specific operations.

We adopt a per-kernel modeling approach, training a separate MLP for each kernel category. Each MLP’s training dataset is built by profiling the corresponding kernel’s execution across various GPU architectures and input parameters. For every sample, we record the actual execution latency on physical hardware as ground-truth.

V. IMPLEMENTATION DETAILS

A. Analytical Models

To ensure our performance model accurately reflects real-world LLM inference workloads, we chose a representative set of critical kernels directly from the backends of popular high-performance serving frameworks, such as SGLang [59] and vLLM [72]. The key characteristics of these kernels are summarized in Table V. Note that for categories like GEMM and Attention, multiple implementations often exist. For cuBLAS

TABLE V
KEY CHARACTERISTICS OF THE KERNELS SELECTED.

Category	Source	Language	Precision	Scheduler	Math Pipe
GEMM	cuBLAS	Pre-compiled	BF16/FP16	HW/SW	Tensor
Scaled MM	vLLM	CUDA C++	FP8	HW/SW	Tensor
Attention	FlashInfer	CUDA C++	BF16/FP16	HW/SW	Tensor, XU
RMSNorm	FlashInfer	CUDA C++	FP32	HW	FMA, XU
SiLU&Mul	FlashInfer	CUDA C++	FP32	HW	FMA, XU
Fused MoE	SGLang	Triton	BF16/FP16	HW	Tensor

GEMM kernels, we observe that specific implementations vary across hardware architectures. For FlashInfer Attention kernels, our analysis includes both FlashAttention-2 (FA2) and FlashAttention-3 (FA3) variants, covering implementations for paged and ragged KV cache layouts [15].

For each kernel category, the implementation of the **Kernel Decomposer** is concise, requiring just 10-50 lines of code. Except for cuBLAS GEMM whose decomposition is taken directly from profiling data, the mapping function $\mathcal{F}$ in Equation (1) for other kernels is drawn from their source code. Because cuBLAS GEMM is closed-source and its implementation differs across hardware architectures, its decomposition behavior is unknown on new GPUs. Therefore, for unseen GPUs lacking profiling data on closed-source kernels, we use decomposition logic from the most architecturally similar GPUs available in our profiling dataset.

Following kernel decomposition, the **Scheduling Simulator** allocates tasks across SMs. For the majority of kernels analyzed (Table V), which utilize the hardware-based scheduler, we simulate the widely inferred RR policy as described in Section IV-B. For cuBLAS GEMM and FlashInfer FA3 kernels [75] on the Hopper architecture, both using persistent kernel designs, we model their respective software-based schedulers. Taking FlashInfer FA3 as an example, we accurately replicated its MinHeap-based scheduler logic in our simulator with around 40 code lines.

The **Feature Analyzer** converts the task distribution into a comprehensive feature set. For math pipelines, our implementation focuses on three types of instruction pipelines most critical to LLM workloads: the Tensor, FMA, and XU pipelines. We found that these three together cover most computational demands in the target kernels. Other pipelines, such as ALU handling logic operations [53], were left out due to their generally low utilization in the kernels and the difficulty in analytically counting their operations.

B. Dataset Construction

To train and evaluate PIPEWEAVE, we built a comprehensive dataset by profiling selected kernels (Table V) across various NVIDIA GPU architectures. The dataset covers 11 different GPU models [36]–[38], [44], representing multiple architectures and market segments. As shown in Table VI, these were split into two groups: the first group was used for training, while the second group was reserved solely for testing to assess PIPEWEAVE’s generalizability to unseen hardware.

Profiling was performed in a consistent software environment using PyTorch 2.8.0, CUDA Toolkit 12.8, FlashInfer

TABLE VI
KEY SPECIFICATIONS OF THE EVALUATED NVIDIA GPUS.

GPU	Architecture	SMs	Mem BW (GB/s)	Tensor BF16 (ops/clk/SM)	Freq (MHz)
A40	Ampere	84	696	1024	1740
A100	Ampere	108	2039	2048	1410
RTX 6000 Ada	Ada	142	960	1024	2505
L20	Ada	92	864	516	2520
H20	Hopper	78	4023	1024	1830
H800	Hopper	132	3352	4096	1830
RTX A6000	Ampere	84	768	1024	1800
L40	Ada	142	864	512	2490
H100	Hopper	132	3352	4096	1830
H200	Hopper	132	4917	4096	1830
RTX PRO 6000 S	Blackwell	188	1792	1024	2340

0.4.1, SGLang 0.5.4, vllm 0.11.0, and Triton 3.4.0. For each combination of kernel, input parameters, serving framework, and GPU hardware, we measured execution latency with the PyTorch Profiler. We conducted 5 warm-up runs followed by 10 measurement runs, using their average as the ground-truth. The profiling dataset includes 6 key kernels serving as core computational backend for vLLM [72] and SGLang [59]:

- **Attention:** 104,958 samples (71,969 training and 32,989 test). $bs \in [1, 16]$, $nh \in [2, 128]$, $nk \in [1, 8]$, $hd \in \{64, 128\}$, $qlen \in [1, 20097]$, $kvlen \in [4, 20481]$. The Query and KV lengths vary randomly within each batch to simulate realistic variable-length sequence patterns.
- **GEMM:** 613,263 samples (494,463 training and 118,800 test). $M \in [2, 131072]$, $N \in [384, 152064]$, $K \in [256, 53248]$.
- **RMSNorm:** 65,036 samples (44,592 training and 20,444 test). $seq \in [2, 131072]$, $dim \in [128, 16384]$.
- **SiLU&Mul:** 104,834 samples (71,868 training and 32,966 test). $seq \in [2, 131072]$, $dim \in [768, 106496]$.
- **Scaled MM:** 25,228 samples (16,818 training and 8410 test). $M \in [2, 131072]$, $N \in [384, 8192]$, $K \in [256, 8192]$.
- **Fused MoE:** 33,264 samples. $M \in [2, 8192]$, $E \in [8, 128]$, $topk \in [2, 8]$, $H \in [1024, 4096]$, $N \in [512, 3072]$. This kernel is used as a detailed case study for our optimization approach in Section VII.

C. MLP Model Training

As outlined in Section IV-D, a separate MLP is trained for each kernel type using derived analytical features. The MLP has a shallow architecture with 3 hidden layers (256, 128, and 64 units), employing ReLU activations followed by Batch Normalization and Dropout (rate 0.1) for regularization. The output layer utilizes a Sigmoid activation to limit predictions to the range $[0, 1]$, representing the kernel’s *execution efficiency* (defined as the ratio of theoretical execution time to actual latency). The final latency prediction is obtained by dividing the theoretical execution time by this estimated efficiency.

Training uses the dataset described in Section V-B. The AdamW optimizer [32] is applied with a 0.001 initial learning rate and weight decay. Mean Absolute Percentage Error (MAPE) serves as the loss function, minimizing relative prediction error. Early stopping is employed to prevent overfitting by monitoring validation loss.

D. End-to-end Performance Prediction

Beyond predicting single kernel performance, we validate our framework’s accuracy in modeling end-to-end LLM inference latency. We built a **Workload Generator** based on the model definitions and kernel invocation logic from both SGLang [59] and vLLM [72]. Given a model configuration and input parameters, this generator creates a sequence of kernel invocations that represents a real inference scenario. Following prior work [26], [76], [80], we assume sequential kernel execution without overlap. For each kernel in the sequence, we use PIPEWEAVE to predict its runtime based on type and input dimensions. The total end-to-end latency for single-GPU inference is calculated by summing all predicted kernel durations.

Predicting end-to-end performance in distributed settings requires modeling both computational kernels and communication kernels required for multi-GPU parallelism [1], [26], [73], [78]. Depending on the employed parallelism, this includes kernels such as *All-Reduce* for Tensor Parallelism (TP) or *Send/Recv* primitives for Pipeline Parallelism (PP). To model these communication kernels, we use a simplified method. We profile their performance across different network topologies and communication volumes to build a baseline performance database. Using this data, we apply a data-driven regression technique (e.g., Random Forest) to estimate communication kernel latency. This prediction is then combined with computational kernel estimates to forecast the total end-to-end latency for distributed inference.

VI. EVALUATION

A. Baselines

To comprehensively evaluate PIPEWEAVE, we conduct our primary evaluation by comparing its prediction accuracy against four main baselines: (1) the classic analytical Roofline model [74]; (2) a Linear regression-based model [29]; (3) Habitat [76]; and (4) Neusight [26], a state-of-the-art data-driven method. To ensure a fair comparison among these primary baselines, we adjusted them to incorporate our analytical components. The Linear model, following the approach in the original paper [29], was trained using two main features from our Feature Analyzer (Section IV-C): theoretical cycles for aggregating compute and memory demand. Similarly, we supplied Habitat and Neusight with the exact task definitions from our Kernel Decomposer (Section IV-A).

Furthermore, to highlight the advantages of our analytical-ML hybrid design in both prediction accuracy and simulation efficiency, we introduce a secondary set of baselines representing highly detailed modeling paradigms: AMALI [6], an instruction-trace-based analytical model, and LLMCompass [78], a hybrid framework that integrates analytical models and cycle-accurate systolic array modeling. Since these detailed simulators provide limited support for diverse modern kernels and incur substantial runtime overhead for end-to-end LLM workloads, we restrict this comparison to standalone GEMMs.

TABLE VII
MAPE (%) OF PIPEWEAVE’S ANALYTICAL OPERATION COUNTS

Metric	gemm8	gemm9	FA2	FA3
Max SM Ops (%)	0.07	0.04	6.34	0.45
Total Ops (%)	0.01	0.14	0.50	0.00

B. Validation of Analytical Components

We first validate PIPEWEAVE’s core analytical components: *Kernel Decomposer*, *Scheduling Simulator*, and *Feature Analyzer*. This step is essential since these parts work in sequence. Any error may propagate and reduce the final feature quality.

We start by verifying the correctness of the *Kernel Decomposer*. Specifically, we compare the number of CTAs from our decomposition process with the ground-truth configurations in the dataset across multiple kernels. The results are fully consistent, confirming decomposition accuracy.

Next, we assess the accuracy of the *Scheduling Simulator* and *Feature Analyzer*. Our method compares analytically derived math pipeline operation counts, both total (kernel-wide) and per-SM maximum operations, against ground-truth measurements from the NVIDIA Nsight Compute (NCU) tool [53]. Due to high profiling overhead and restricted hardware access, we perform this validation on two flagship devices: A100 and H100. The evaluation covers four key kernel implementations: cuBLAS GEMM (gemm8 on A100 and gemm9 on H100), FA2, and FA3. Each includes about 500 test samples randomly sampled from the workload configuration ranges defined in Section V-B. As shown in Table VII, our model achieves a maximum error of **0.5%** for total operations and **6.3%** for the maximum per-SM operations. The higher error for FA2 (6.34%) relative to FA3 (0.45%) is mainly due to their different scheduling mechanisms: FA3 uses a persistent-kernel design with deterministic task scheduling that can be explicitly simulated, whereas FA2 relies on dynamic hardware scheduling, which introduces additional uncertainty in predicting peak per-SM workload.

Finally, we conduct an ablation study on the GEMM and Attention kernels using their full datasets (Section V-B) to highlight the contribution of our core components. We compare the full PIPEWEAVE model against three ablated variants: (1) w/o MIO (without MIO features), (2) w/o Math, and (3) w/o MLP (replacing MLP with a Roofline-based predictor). As shown in Figure 4, each component is crucial for accurate performance. For the Attention kernel, the full model achieves **1.1×**, **1.8×** and **2.9×** higher accuracy than w/o MIO, w/o Math, and w/o MLP respectively. The effect is stronger for GEMM, where our full model improves accuracy by **3.2×** (w/o MIO), **2.7×** (w/o Math), and **3.5×** (w/o MLP), respectively. While both kernels benefit significantly from our modeling framework, the final prediction error for Attention kernels (15.54%) remains higher than that of GEMM kernels (8.39%). As previously shown in Table VII, this gap is not caused by inaccuracies in the analytical operation counts, which remain comparably low for both kernels. Instead, it arises from the inherently uneven workload distribution and dynamic execu-

TABLE VIII
PREDICTION ERROR ON SEEN AND UNSEEN GPUS.

Hardware	Roofline	Linear	Habitat	Neusight	PIPEWEAVE
Seen	72.22%	59.50%	28.92%	43.49%	6.77%
Unseen	79.61%	70.28%	85.96%	46.70%	13.14%

tion characteristics of Attention mechanisms. Unlike GEMM, where tasks are defined by uniform dimensional parameters across tiles, Attention workloads exhibit substantial variance. This variance primarily results from causal masking—where tasks processing earlier query tokens attend to fewer key/value tokens than those handling later tokens—as well as randomly varying sequence lengths within a batch. In addition, Attention introduces more complex memory behavior and heterogeneous execution phases with different compute–memory characteristics, which further increase runtime variability. These factors make execution latency more sensitive to hardware scheduling dynamics and lead to larger inter-block completion variance. Consequently, Attention latency is inherently more difficult for the MLP to model than the stable and uniform execution patterns observed in GEMM workloads.

C. Kernel-Level Prediction Accuracy

We evaluate PIPEWEAVE on a dataset of around **1M** samples from 11 different GPUs (Section V-B). This dataset includes fundamental kernels from modern inference frameworks such as vLLM [72] and SGLang [59]), covering FP32, BF16/FP16, and FP8 precisions. PIPEWEAVE achieves state-of-the-art prediction accuracy and significantly surpasses prior work. On seen hardware, it attains an average MAPE of 6.0%, outperforming the next-best Neusight at 42.6%. On unseen hardware, our framework demonstrates superior generalization with an average MAPE of 11.5%—a **3.9×** improvement compared to Neusight (45.1%).

Figure 5 shows the prediction accuracy (MAPE) for four typical kernels in BF16 LLM inference scenarios. Correspondingly, Table VIII summarizes the average MAPE across these four kernels on both seen and unseen hardware. Errors for Linear and Roofline models are significantly higher than PIPEWEAVE on both seen and unseen hardware, with peak MAPEs reaching 215.6% and 263.5%, respectively. Although the SOTA baseline, Neusight, outperforms other base-

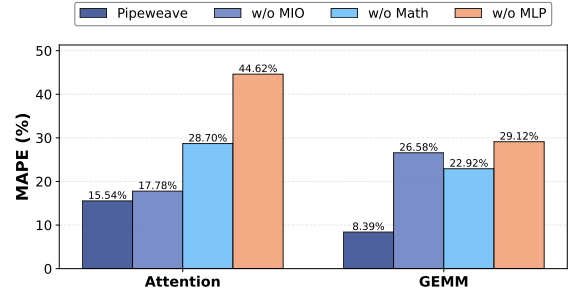


Fig. 4. Ablation study on the impact of MIO and Math Pipeline features for GEMM and Attention kernels.

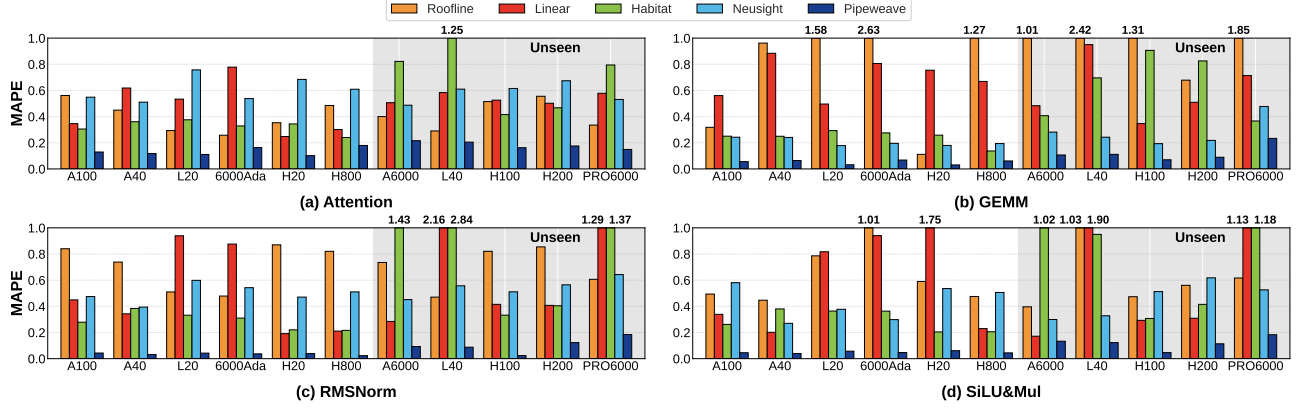


Fig. 5. Kernel-level prediction accuracy (MAPE) of PIPEWEAVE and baseline models. Unseen hardware platforms are identified by a grey background.

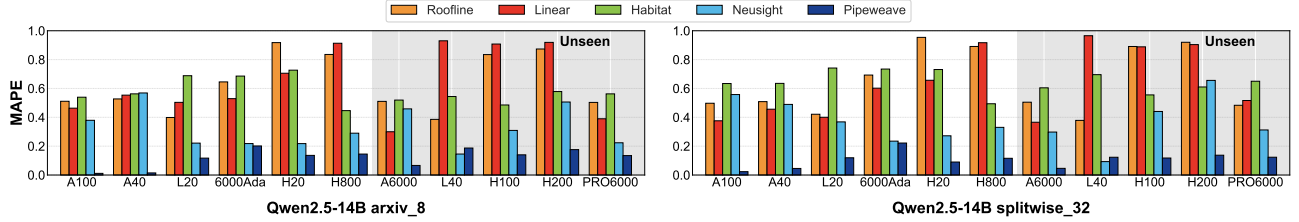


Fig. 6. End-to-end inference prediction accuracy (MAPE) of PIPEWEAVE and baseline models for single-GPU Qwen2.5-14B inference using SGLang. Unseen hardware platforms are identified by a grey background.

lines, its highest error of 75.7% remains substantially above PIPEWEAVE’s 23.4%. Furthermore, the prediction errors of analytical approaches, namely the Linear and Roofline models, are highly hardware-dependent. For instance, Figure 5(b) highlights a stark contrast in the Roofline model’s MAPE for GEMM kernels between the H20 (11%) and H800 (127%). This difference arises from the distinct compute-to-memory ratios of the two GPUs. Specifically, while the H20 retains approximately 120% of the H800’s memory bandwidth, its peak compute capability is restricted to roughly 15% of the H800’s. Under this extremely low compute-to-memory ratio, the compute units on the H20 are easily saturated. The abundant memory bandwidth ensures that execution pipelines are constantly fed, allowing GEMMs to sustain throughput very close to the theoretical peak; thus, the Roofline estimate remains accurate. Conversely, the H800 features a massive compute capacity that is exceedingly difficult to fully saturate in most practical scenarios, as reaching the theoretical peak requires near-perfect instruction-level concurrency and uninterrupted data delivery. In practice, inevitable microarchitectural frictions prevent kernels from approaching this idealized Roofline peak, leading to significant overestimation. Unlike such models, PIPEWEAVE’s MLP naturally learns these hardware-specific inefficiencies, thereby achieving significantly lower prediction errors.

Besides the four kernels common in BF16 LLM inference scenarios, we also trained and tested the scaled mm kernel (block-wise quantization) for FP8 inference on the Hopper architecture, achieving high prediction accuracy. On seen hardware (H20, H800), PIPEWEAVE’s MAPE was 1.9% and 4.1%, while on unseen hardware (H100, H200), MAPE was

4.2% and 5.2%. This highlights the framework’s adaptability to FP8 precision kernels, achieving average accuracy gains of **10.8×**, **9.5×**, **5.5×**, and **7.8×** over Roofline, Linear, Habitat, and Neusight.

Finally, to evaluate PIPEWEAVE’s prediction accuracy and simulation efficiency, we conduct a targeted comparison with AMALI and LLMCompass on an A100 GPU. As outlined in our baseline methodology (Section VI-A), this comparison is restricted to GEMMs due to the high computational overhead of these detailed simulators. Using 540 distinct GEMM samples with varying dimensions randomly drawn from our dataset (Section V-B), we measure prediction error and per-GEMM simulation overhead. Figure 7 shows the comparison results, where prediction error is reported as signed relative error to capture both over- and under-estimation. Overall, PIPEWEAVE achieves substantially lower simulation overhead while maintaining higher prediction accuracy. On average, it obtains a MAPE of **6.4%**, compared with **28.3%** for AMALI and **29.7%** for LLMCompass, while reducing prediction time by **3 to 7 orders of magnitude**. These results indicate that the grey-box design—combining pipeline-demand analytical modeling with ML—can effectively capture dominant performance factors without requiring expensive low-level simulation.

D. E2E Inference Accuracy

Beyond kernel-level validation, we assess PIPEWEAVE’s end-to-end predictive accuracy by comparing its simulations with actual serving latencies from SGLang [59] and vLLM [72]. Following prior work [1], we use two representative datasets Arxiv Summarization [8] and Splitwise [55]—and test three typical LLMs (Qwen2.5-14B, Qwen3-32B, and

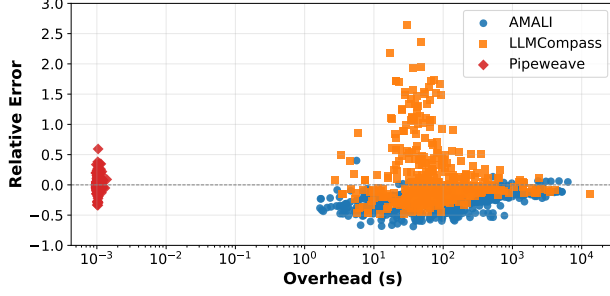


Fig. 7. Comparison of simulation overhead versus relative prediction error for GEMM workloads on the A100 GPU.

Llama3.1-70B) in both single-GPU (TP=1) and distributed (TP, PP) inference settings.

Workloads for these datasets are generated by randomly sampling requests to create batches of varying sizes, such as `arxiv_8` and `splitwise_64`. The `arxiv_*` (where `*` denotes the batch size) workloads have an average input length of 2,630 tokens, while the `splitwise_*` workloads average 982 tokens. Output lengths vary from 5 to 4,056 tokens.

For single-GPU (TP=1) evaluations, we tested Qwen2.5-14B across all 11 GPUs (Figure 6). PIPEWEAVE achieves an average MAPE of 11.3%, notably outperforming the best baseline, Neusight at 34.5%. Furthermore, PIPEWEAVE maintains high accuracy on unseen GPUs, with a 12.5% MAPE—a significant **2.8×** improvement over Neusight’s 34.4%.

This robustness extends to distributed inference. As shown in Table IX, PIPEWEAVE delivers consistent accuracy across diverse end-to-end inference scenarios. It spans two inference frameworks (SGLang and vLLM), multiple models (Qwen3-32B and Llama3.1-70B), and various parallelism strategies (TP=2, TP=4, TP=8, and TP=4&PP=2). PIPEWEAVE consistently achieves low MAPE averages: 8.4% (SGLang, Qwen3-32B, TP=2), 4.3% (SGLang, Llama3.1-70B, TP=4), 7.7% (SGLang, Llama3.1-70B, TP=8), and an excellent 4.0% (vLLM, Llama3.1-70B, TP=4&PP=2). This performance significantly surpasses the best baseline Neusight. Across all 20 tested configurations, PIPEWEAVE achieves an overall average MAPE of 6.6% versus Neusight’s 34.7%, showing a **5.3×** average accuracy improvement. Interestingly, our analysis shows that in some E2E inference scenarios, baselines such as Neusight can exhibit very low E2E errors (e.g., 0.5%) despite having poor kernel-level prediction accuracy. We identify two primary causes for this phenomenon. First, E2E latency aggregates the execution time of many kernels, which leads to systematic error cancellation: overestimations for some kernels offset underestimations for others, thereby reducing the overall E2E error. Second, E2E inference typically involves a much narrower and more constrained set of workload dimensions than those covered in comprehensive kernel-level evaluations (Section V-B); consequently, these workloads often lie near the baseline’s prediction “sweet spots.”

In summary, PIPEWEAVE delivers **high fidelity, fast pre-**

diction, and broad generalizability for GPU performance modeling.

VII. BEYOND SIMULATION

In prior sections, we verified the robustness of PIPEWEAVE. Trained with a MAPE loss, our framework demonstrates strong accuracy in forecasting the performance of various *well-optimized* kernels on diverse hardware platforms. In this section, we transition from general prediction to a more challenging task: **optimization guidance**. Our goal is to improve the performance of the Fused MoE Triton kernel—the default MoE backend in SGLang [59]—across hardware platforms.

The primary challenge lies in the opacity of performance potential. For any given input shape and hardware platform, the attainable performance ceiling is unknown. Consequently, we cannot determine *a priori* whether a current execution is near-optimal or sub-optimal. For instance, achieving 50% of the roofline [74] on an A40 might be poor if the true ceiling is 70%, while 20% on an A100 could be near-optimal if the ceiling is only 21%. Lacking this ground truth, it is impossible to systematically quantify the performance gap or identify where system-level optimization efforts should be directed.

Therefore, we look beyond simulation. Rather than predicting average performance, our aim is to provide practical optimization guidance. We aim to address the following questions:

- (1) Can modeling help establish the kernel’s true “Potential Performance Ceiling”, distinct from the noise of sub-optimal configurations?
- (2) Can this estimated “ceiling” serve as a reference to identify systematic underutilization and guide optimization efforts?

A. Defining the Potential Ceiling via Quantile Loss

To address this issue, we adopt the principles of Quantile Regression [23]. We train an MLP model using the same feature set and target (execution efficiency) described in Section V-C, but employ Quantile Loss as the training objective. We specifically configure the model to predict the 80th percentile (P80). This approach provides a statistically robust estimate of the performance ceiling, which is less sensitive to extreme outliers or measurement noise compared to higher quantiles such as P90.

By targeting P80, the model is effectively trained to fit the top 20% of performance data points, capturing the characteristics of high-performing configurations while systematically filtering out the lower 80% of sub-optimal results. Consequently, the resulting prediction, $\hat{y}_{p80}$, does not represent a typical average. Instead, it serves as a statistically-defined **Potential Performance Ceiling**, representing a high yet realistically achievable target for the kernel’s implementation.

B. Diagnosing the Performance Gap

We first validate the P80 model as a diagnostic tool. The trained model, which predicts the P80 ceiling $\hat{y}_{p80}$, is applied across the entire Fused MoE dataset (Section V-B). We then

TABLE IX
END-TO-END PERFORMANCE PREDICTION MAPE (%) OF PIPEWEAVE AND BASELINES FOR MULTI-GPU INFERENCE USING SGLANG AND VLLM.

Framework	Model	Dataset	Hardware	Roofline	Linear	Habitat	Neusight	PIPEWEAVE
SGLang	Qwen3-32B (TP=2)	arxiv_l2	A100	48.6	42.7	47.3	45.0	2.4
			6000Ada	59.2	43.3	44.9	30.4	9.1
			H100	73.5	77.1	34.9	31.1	7.5
			PRO6000	46.5	15.2	39.6	56.6	9.3
		splitwise_48	A100	49.0	44.6	35.5	45.9	3.9
			6000Ada	53.2	51.5	35.1	38.8	7.9
			H100	62.4	49.6	33.3	60.2	16.5
			PRO6000	47.1	29.8	36.9	18.5	10.9
SGLang	Llama3.1-70B (TP=4)	arxiv_l6	A100	45.2	30.3	50.1	76.5	2.6
			H100	78.6	69.4	45.6	34.5	5.4
		splitwise_64	A100	46.0	26.2	57.5	55.6	2.1
			H100	82.2	64.8	56.1	47.2	7.0
SGLang	Llama3.1-70B (TP=8)	arxiv_l6	H20	90.1	70.5	54.4	27.1	4.0
			H800	66.7	46.7	25.9	17.2	12.3
		splitwise_64	H20	91.8	74.3	62.7	20.4	3.7
			H800	69.8	51.5	29.1	26.1	10.7
vLLM	Llama3.1-70B (TP=4,PP=2)	arxiv_l6	H20	69.1	45.2	54.6	0.5	3.0
			H800	25.7	60.8	9.0	16.9	0.7
		splitwise_64	H20	76.7	64.7	72.6	19.1	2.3
			H800	49.5	67.1	38.6	23.7	9.9

measure the *Performance Gap* by computing the difference between the predicted ceiling and the actual performance:

$$\text{perf_gap} = \hat{y}_{p80} - y_{\text{actual}}$$

Here, y_{actual} represents execution efficiency (Section V-C).

Figure 8 presents a consolidated analysis of these gaps. Each vertical bar represents a hardware platform, with the bar height indicating the total number of identified underperforming points on that platform. The line plot shows the cumulative distribution function (CDF) of the performance gaps aggregated across all evaluated hardware platforms. The figure reveals two key findings. First, the CDF line confirms a “long tail” pattern. We observe that while the vast majority of configurations perform near their potential, approximately 80% of all points have a Performance Gap below 0.1. Based on this observation, we identify an **“Underperforming Point”** as any configuration where the Performance Gap > 0.1 . Second, the bar chart pinpoints where these Underperforming Points occur, revealing that significant inefficiencies are hardware-specific. For instance, the A40 GPU exhibits the largest discrepancies, accounting for the vast majority of inefficiencies with **921** distinct Underperforming Points (representing **30.4%** of all A40 samples). This clearly indicates that the kernel’s current configuration logic is ill-suited for this specific hardware architecture. In stark contrast, the H20 achieves near-optimal results, exhibiting **zero** such points.

C. Closing Performance Gap by Tuning Parameters

In Section VII-B, we apply our P80 model to successfully identify “Underperforming Points”. We now verify that these gaps are actionable and indicative of systemic optimization potential. Approximately 70 unique “Underperforming Point” configurations are selected for each GPU: A40, L20, A100, and H800. For these targeted cases, optimization is

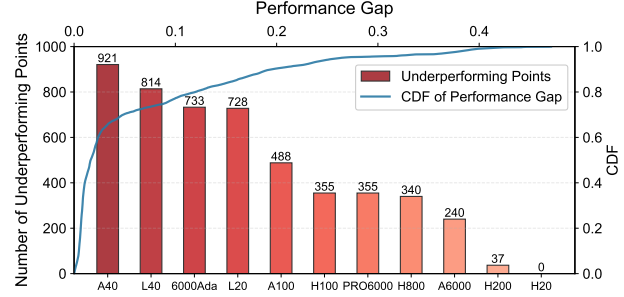


Fig. 8. Performance Gap analysis. The CDF of the gap distribution (line) and the count of “Underperforming Points” (Gap > 0.1) by hardware (bars).

conducted via brute-force autotuning over three parameters: BLOCK_SIZE, num_stages, and num_warps.

To explicitly validate our statistical diagnostic methodology against actual optimization outcomes, Table X shows the relationship between the systemic density of underperforming points and the achieved tuning benefits. A clear positive correlation is observed (Pearson correlation coefficient of **0.86**): hardware platforms with a higher count of underperforming points obtain larger geometric mean speedups after tuning. This result confirms that our statistical diagnosis effectively reflects real optimization potential and can guide tuning efforts toward configurations with the largest expected gains.

TABLE X
SPEEDUP VS. UNDERPERFORMING POINTS ACROSS GPUS.

GPU	Underperforming Points	Geo-mean Speedup
A40	921	1.61×
L20	728	1.12×
A100	488	1.06×
H800	340	1.03×

Furthermore, Figure 9 demonstrates the practical impact of these diagnosed underperforming points. After applying brute-

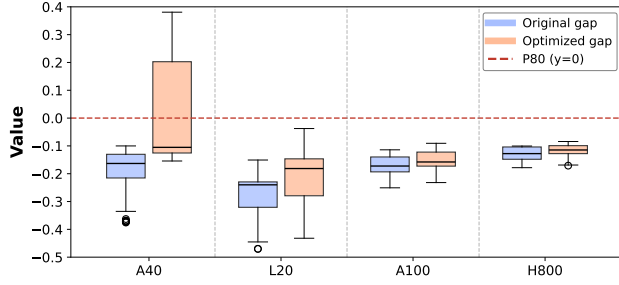


Fig. 9. Performance gap distribution before and after model-guided optimization across four GPU platforms.

force autotuning, the average performance gap is noticeably reduced, particularly on hardware that initially exhibits larger inefficiencies. For example, the average gap on A40 decreases from **0.187** to **0.083**, and on L20 from **0.274** to **0.215**. In contrast, the improvements on A100 and H800 are more limited, as their baseline configurations are already closer to the estimated performance ceiling. Despite these improvements, a residual gap often remains. This suggests that certain inefficiencies cannot be fully eliminated through parameter tuning alone, but may instead stem from deeper factors such as the kernel’s structural design or inherent limitations of the Triton programming model [12], [58].

VIII. RELATED WORK

A. GPU Performance Modeling

Research on GPU performance modeling is broadly divided into three categories: cycle-accurate simulators [4], [22], [66], analytical models [6], [19], [25], [78], and data-driven approaches [26], [76]. Despite their usefulness, these approaches present inherent trade-offs. High-fidelity cycle-accurate simulators are computationally expensive and difficult to generalize across new hardware. Faster alternatives—analytical and data-driven models—often face limited accuracy, hardware-specific constraints, and coarse-grained assumptions that miss complex behaviors such as fused-kernel coupling, restricting their generalization. PIPEWEAVE is designed to address these limitations by combining principled analytical modeling with the speed and flexibility of data-driven techniques, enabling high fidelity and broad generalization.

Table XI summarizes representative GPU performance models. Unlike prior methods relying on empirical black-box learning or coarse-grained analytical abstractions (e.g., tile-level throughput and static wave scheduling), PIPEWEAVE advances the grey-box paradigm via a microarchitecture-aware, pipeline-level formulation. By explicitly capturing heterogeneous pipeline demands and dynamic scheduling, it achieves high accuracy and cross-architecture portability.

B. Network Simulation

As computation scales across multi-node clusters, precise modeling of data center interconnects grows increasingly vital. General-purpose network simulators [17], [69] provide granular packet-level control to evaluate congestion, routing

TABLE XI
COMPARISON OF MICROARCHITECTURAL MODELING CAPABILITIES.

Dimension	Habitat	Neusight	PIPEWEAVE (Ours)
Modeling Strategy	Black-box	Macro Grey-box	Micro-arch Grey-box
Granularity	Kernel-level	Tile-level	Pipeline-level
Hardware Fidelity	GPU	SM	Pipeline
Scheduling Semantics	N/A	Static wave assumption	Dynamic SM scheduling
Kernel Type	Elemental kernels	Elemental kernels	Fused & Elemental kernels
Cross-Arch Generality	Low	Medium	High
Prediction Accuracy	Low	Medium	High

behavior, and protocol interactions. Newer AI-focused network simulation frameworks [57], [73] target communication patterns such as All-Reduce and All-Gather, as well as the performance characteristics of large-scale, communication-heavy distributed workloads.

C. System-Level Simulation

Beyond individual components, extensive research focuses on end-to-end DNN system performance simulation. These tools model the complex interactions among computation, communication, and scheduling strategies for large models. In distributed training, simulators [5], [57], [73] evaluate various parallelism strategies, such as data, pipeline, tensor. For inference, particularly in LLM serving, tools [1], [7], [14] simulate dynamic batching and scheduling policies. Our PIPEWEAVE framework not only incorporates a system-level simulator for inference, but also offers a high-fidelity, pluggable GPU computation model required by prior system-level tools.

IX. CONCLUSION

We present PIPEWEAVE, a unified framework that synergizes knowledge-guided analytical modeling with data-driven learning to achieve high-fidelity GPU performance prediction. By decomposing kernels into fundamental pipeline demands and capturing complex runtime interactions via an MLP, PIPEWEAVE demonstrates state-of-the-art accuracy and generalization across diverse kernels, workloads, and hardware generations. Beyond prediction, we validated its practical utility in diagnosing hardware-specific inefficiencies and guiding targeted optimizations.

Future work will focus on two main areas. First, we will extend PIPEWEAVE to complex distributed settings, incorporating support for multi-node clusters and advanced parallelism strategies such as Expert Parallelism (EP). Second, we plan to broaden our model-guided optimization method by developing automated tools that detect performance bottlenecks and enhance configuration logic for more production kernels.

X. ACKNOWLEDGMENTS

We thank the anonymous reviewers for their constructive feedback and valuable suggestions that improved this work. We also thank our colleagues for helpful discussions. This work used LLMs for text refinement and code generation. This work was supported by Alibaba Tech infra and Reliability Engineering (TRE) in Alibaba Group through Alibaba Innovative Research Program and Alibaba Research Intern Program.

REFERENCES

- [1] A. Agrawal, N. Kedia, J. Mohan, A. Panwar, N. Kwatra, B. S. Gulavani, R. Ramjee, and A. Tumanov, "Vidur: A large-scale simulation framework for llm inference," in *Proceedings of the 2024 Conference on Machine Learning and Systems (MLSys '24)*, 2024, also available at arXiv:2405.05465. [Online]. Available: <https://arxiv.org/abs/2405.05465>
- [2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. Santa Clara, CA, USA: USENIX Association, 2024, also available on arXiv:2403.02310. [Online]. Available: <https://arxiv.org/abs/2403.02310>
- [3] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, X. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023, arXiv:2309.16609.
- [4] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2009, pp. 163–174.
- [5] J. Bang, Y. Choi, M. Kim, Y. Kim, and M. Rhu, "vTrain: A simulation framework for evaluating cost-effective and compute-optimal large language model training," in *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO 2024)*. IEEE / ACM, 2024, pp. 153–167. [Online]. Available: <https://arxiv.org/abs/2312.12391>
- [6] S. Cao, J. Wu, J. Chen, H. An, and Z. Yu, "Amali: An analytical model for accurately modeling llm inference on modern gpus," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. ACM, 2025, pp. 1495–1508.
- [7] J. Cho, M. Kim, H. Choi, G. Heo, and J. Park, "Llmservingsim: A hw/sw co-simulation infrastructure for llm inference serving at scale," in *2024 IEEE International Symposium on Workload Characterization (IISWC)*, 2024, pp. 1–12.
- [8] A. Cohan, F. Dernoncourt, D. S. Kim, T. Bui, S. Kim, W. Chang, and N. Goharian, "A discourse-aware attention model for abstractive summarization of long documents," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 2 (Short Papers)*. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 615–621. [Online]. Available: <https://aclanthology.org/N18-2097/>
- [9] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, Z. Xie, Y. K. Li, P. Huang, F. Luo, C. Ruan, Z. Sui, and W. Liang, "Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 1280–1297. [Online]. Available: <https://aclanthology.org/2024.acl-long.70>
- [10] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] J. H. Davis *et al.*, "Taking gpu programming models to task for performance: an empirical study," in *Proceedings of ICS 2025*, 2025, demonstrates that abstraction and language-level limitations cause persistent, architecture-dependent performance gaps. [Online]. Available: https://hpcrl.github.io/ICS2025-webpage/program/Proceedings_ICS25/ics25-63.pdf
- [13] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, pp. 1–39, 2022.
- [14] Y. Feng, X. Tan, K. H. Sew, Y. Jiang, Y. Zhu, and H. Xu, "Simulating the next generation of llm inference systems," in *Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25)*. ACM, 2025.
- [15] FlashInfer Team, "Kv cache layout tutorial," https://docs.flashinfer.ai/tutorials/kv_layout.html, 2025, accessed: 2025-10-27.
- [16] Google DeepMind, "Gemini 2.5: Expanding the Capabilities of Multimodal AI Models," <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>, 2025, accessed: Nov. 2025.
- [17] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, "Network simulations with the ns-3 simulator," in *SIGCOMM Demonstration*, 2008. [Online]. Available: <https://www.nsnam.org/>
- [18] S. Hong and H. Kim, "An analytical model for a gpu architecture with memory-level and thread-level parallelism awareness," in *Proceedings of the 36th Annual International Symposium on Computer Architecture (ISCA '09)*. ACM, 2009, pp. 152–163.
- [19] J.-C. Huang, J. H. Lee, H. Kim, and H.-H. S. Lee, "Gpumech: Gpu performance modeling technique based on interval analysis," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 2014, pp. 268–279.
- [20] Y. Ji, W. Li, X. Shen, and X. Shen, "Dynamic thread block scheduling for gpu-based computing," in *Proceedings of the 22nd International Conference on Parallel Architectures and Compilation Techniques (PACT '13)*. IEEE, 2013, pp. 375–386.
- [21] A. Jog, P. Nadkarni, O. Kayiran, R. Das, M. Kandemir, O. Mutlu, V. Narayanan, and C. R. Das, "Owl: Cooperative thread array aware scheduling techniques for improving gpgpu performance," in *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA '16)*. IEEE, 2016, pp. 395–406.
- [22] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE/ACM, 2020, pp. 473–486.
- [23] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
- [24] A. Kuzmin, M. van Baalen, Y. Ren, M. Nagel, J. Peters, and T. Blankevoort, "Fp8 quantization: The power of the exponent," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/hash/5e07476b6bd2497e1fbd11b8f0b2de3c-Abstract-Conference.html
- [25] J. Lee, Y. Ha, S. Lee, J. Woo, J. Lee, H. Jang, and Y. Kim, "Gcom: a detailed gpu core model for accurate analytical modeling of modern gpus," in *Proceedings of the 49th Annual International Symposium on Computer Architecture (ISCA '22)*. Association for Computing Machinery, 2022, pp. 424–436.
- [26] S. Lee, A. Phanishayee, and D. Mahajan, "Forecasting gpu performance for deep learning training and inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 493–508. [Online]. Available: <https://doi.org/10.1145/3669940.3707265>
- [27] A. H. Less Wright, "Deep Dive on CUTLASS Ping-Pong GEMM Kernel," <https://pytorch.org/blog/cutlass-ping-pong-gemm-kernel/>, November 2024, accessed: 2025-10-18.
- [28] A. Li, S. L. Song, W. Liu, X. Liu, A. Kumar, and H. Corporaal, "Locality-aware cta clustering for modern gpus," in *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '17)*. Xi'an, China: ACM, 2017, pp. 297–311. [Online]. Available: <https://doi.org/10.1145/3037697.3037709>
- [29] Y. Li, Y. Sun, and A. Jog, "Path forward beyond simulators: Fast and accurate gpu execution time prediction for dnn workloads," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 380–394. [Online]. Available: <https://doi.org/10.1145/3613424.3614277>
- [30] A. Liu, S. L. Song, W. Liu, A. Kumar, and H. Corporaal, "Greedy dual-size thread block scheduling for gpus," in *Proceedings of the 42nd International Conference on Parallel Processing (ICPP '13)*. IEEE, 2013, pp. 320–329.
- [31] X. Liu, A. Li, J. Yang, A. Nukada, B. Ren, and W.-m. W. Hwu, "Locality analysis for gpgpu programs," in *Proceedings of the International Symposium on Microarchitecture (MICRO '12)*. IEEE, 2012, pp. 63–74.
- [32] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization,"

- in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkg6RiCqY7>
- [33] Modal Labs, “Streaming Assembler (SASS) — GPU Glossary,” <https://modal.com/gpu-glossary/device-software/streaming-assembler>, 2025, accessed: 2025-10-20.
- [34] J. Nickolls, “Gpu parallel computing architecture and cuda programming model,” in *2007 IEEE Hot Chips 19 Symposium (HCS)*, 2007, pp. 1–12.
- [35] NVIDIA Corporation, *NVIDIA CUDA C Programming Guide*, 2009, version 2.3. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [36] —, *NVIDIA Ampere Architecture Whitepaper (GA10x/A100)*, 2020, “NVIDIA A100 Tensor Core GPU Architecture In-Depth” and “NVIDIA Ampere GA102 GPU Architecture” Whitepapers. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-architecture-whitepaper.pdf>
- [37] —, *NVIDIA Ada GPU Architecture Whitepaper (Ada Lovelace)*, 2022, “NVIDIA Ada GPU Architecture” V2.02. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>
- [38] —, *NVIDIA Hopper GPU Architecture Whitepaper (H100 Tensor Core GPU)*, 2022, “NVIDIA H100 Tensor Core GPU Architecture” Whitepaper V1.01. [Online]. Available: <https://advancedclustering.com/wp-content/uploads/2022/03/gtc22-whitepaper-hopper.pdf>
- [39] —, “Cuda gpus,” 2024. [Online]. Available: <https://developer.nvidia.com/cuda-gpus>
- [40] —, *CUTLASS: CUDA Templates for Linear Algebra Subroutines – Scaled Matrix Multiplication*, 2024, version 3.5, Persistent and ScaledMM kernels. [Online]. Available: <https://github.com/NVIDIA/cutlass>
- [41] —, *DeepGEMM: High-Performance FP8 GEMM Kernels for Transformer Inference*, 2024, fp8 GEMM library for Hopper and Ada architectures. [Online]. Available: <https://github.com/NVIDIA/DeepGEMM>
- [42] —, “Efficient gemm in cutlass,” https://docs.nvidia.com/cutlass/media/docs/cpp/efficient_gemm.html, oct 2024, accessed: 2025-10-27. CUTLASS Documentation.
- [43] —, “Matrix multiplication,” <https://docs.nvidia.com/deeplearning/performance/dl-performance-matrix-multiplication/index.html>, oct 2024, accessed: 2025-10-27. Part of the NVIDIA Deep Learning Performance Guide.
- [44] —, *NVIDIA Blackwell Architecture Whitepaper (RTX/AI Data-Center)*, 2024, “NVIDIA RTX Blackwell GPU Architecture” Whitepaper V1.1. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
- [45] —, *Transformer Engine: FP8 Training and Inference*, 2024, version 1.6, Apache License 2.0. [Online]. Available: <https://github.com/NVIDIA/TransformerEngine>
- [46] —, *CUDA C++ Best Practices Guide*, 2025, version 13.0. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [47] —, *CUDA C Programming Guide*, 2025, version 13.0. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [48] —, “CUDA Compiler Driver NVCC Documentation,” <https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/>, 2025, accessed: 2025-10-20.
- [49] —, *CUDA Driver API Documentation*, NVIDIA Corporation, 2025, cUDA Toolkit v13.0.97; last updated Oct 2, 2025. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-driver-api/>
- [50] —, “CUTLASS Documentation,” <https://docs.nvidia.com/cutlass/index.html>, 2025, accessed: 2025-10-18.
- [51] —, “NVIDIA cuBLAS Library Documentation,” <https://docs.nvidia.com/cuda/cublas/>, 2025, accessed: 2025-10-18.
- [52] —, “NVIDIA Developer Forums,” <https://forums.developer.nvidia.com>, 2025, accessed: 2025-10-20.
- [53] —, “NVIDIA Nsight Compute Documentation,” <https://docs.nvidia.com/nsight-compute>, 2025, accessed: 2025-10-20.
- [54] —, “Parallel Thread Execution ISA Version 9.0 Documentation,” <https://docs.nvidia.com/cuda/parallel-thread-execution/>, 2025, accessed: 2025-10-20.
- [55] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, Buenos Aires, Argentina, 2024. [Online]. Available: <https://dl.acm.org/doi/10.1109/ISCA59077.2024.00019>
- [56] PyTorch Team, “Pytorch profiler: Performance analysis tool for deep learning,” <https://pytorch.org/docs/stable/profiler.html>, 2024, accessed: 2025-11-04.
- [57] S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, “Astra-sim: Enabling sw/hw co-design exploration for distributed deep learning training platforms,” in *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2020, pp. 81–92.
- [58] B. Ringlein, T. Parnell, and R. Stoica, “Gpu performance portability needs autotuning,” *arXiv preprint*, 2025, shows that residual performance gaps often stem from fundamental kernel design limits rather than parameter tuning alone. [Online]. Available: <https://arxiv.org/abs/2505.03780>
- [59] SGLang Project, “SGLang: Fast Serving Framework for Large Language Models and Vision-Language Models,” <https://github.com/sgl-project/sglang>, 2024, version 0.5.3, Apache License 2.0.
- [60] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision,” <https://arxiv.org/abs/2407.08608>, July 2024, arXiv:2407.08608 [cs.LG].
- [61] N. Shazeer, “Glu variants improve transformer,” *arXiv preprint arXiv:2002.05202*, 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [62] N. Shazeer *et al.*, “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [63] H. Shen, N. Mellempudi, X. He, Q. Gao, C. Wang, and M. Wang, “Efficient post-training quantization with fp8 formats,” in *Proceedings of the 6th Conference on Machine Learning and Systems (MLSys 2024)*, 2024, arXiv preprint arXiv:2309.14592v2. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/hash/dea9b4b6f55ae611c54065d6fc750755-Abstract-Conference.html
- [64] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 31 094–31 116. [Online]. Available: <https://proceedings.mlr.press/v202/sheng23a.html>
- [65] S. L. Song, A. Li, X. Liu, A. Kumar, and H. Corporaal, “Understanding the impact of cta scheduling on gpu performance,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 6, pp. 1738–1751, 2016.
- [66] Y. Sun, T. Baruah, S. A. Mojmader, S. Dong, X. Gong, S. Treadway, Y. Bao, S. Hance, C. McCardwell, V. Zhao, and *et al.*, “Mgpu-sim: Enabling multi-gpu performance modeling and optimization,” in *Proceedings of the 46th Annual International Symposium on Computer Architecture (ISCA)*. ACM, 2019, pp. 197–209.
- [67] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, Édouard Grave, and G. Lample, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023, arXiv:2302.13971.
- [68] Triton Team, “Triton Language Documentation,” <https://triton-lang.org/main/index.html>, 2025, accessed: 2025-10-20.
- [69] A. Varga and R. Hornig, “An Overview of the OMNeT++ Simulation Environment,” in *Proceedings of the 1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems*, ser. SIMUTOOLS ’08. ICST, 2008, pp. 1–10.
- [70] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 6000–6010.
- [71] A. Vladimirov, “CUTLASS Tutorial: Persistent Kernels and Stream-K,” <https://research.colfax-intl.com/cutlass-tutorial-persistent-kernels-and-stream-k/>, 2024, accessed: 2025-10-18.
- [72] vLLM Project, “vLLM: A High-Throughput and Memory-Efficient Inference and Serving Engine for Large Language Models,” <https://github.com/vllm-project/vllm>, 2025, version 0.11.0 (latest Oct 2 2025), Apache License 2.0.
- [73] X. Wang, Q. Li, Y. Xu, G. Lu, D. Li, L. Chen, H. Zhou, L. Zheng, S. Zhang, Y. Zhu, Y. Liu, P. Zhang, K. Qian, K. He, J. Gao, E. Zhai, D. Cai, and B. Fu, “Simai: Unifying architecture

- design and performance tuning for large-scale large language model training with scalability and precision,” in *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI '25)*. Philadelphia, PA, USA: USENIX Association, 2025, pp. 541–558. [Online]. Available: <https://www.usenix.org/conference/nsdi25/presentation/wang-xizheng-simai>
- [74] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: <https://doi.org/10.1145/1498765.1498785>
- [75] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “Flashinfer: Efficient and customizable attention engine for llm inference serving,” *arXiv preprint arXiv:2501.01005*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [76] G. X. Yu, Y. Gao, P. Golikov, and G. Pekhimenko, “Habitat: A runtime-based computational performance predictor for deep neural network training,” in *USENIX Annual Technical Conference*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:236992542>
- [77] B. Zhang and R. Sennrich, “Root mean square layer normalization,” *CoRR*, vol. abs/1910.07467, 2019. [Online]. Available: <http://arxiv.org/abs/1910.07467>
- [78] H. Zhang, A. Ning, R. B. Prabhakar, and D. Wentzlaff, “Llmcompass: Enabling efficient hardware design for large language model inference,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ser. ISCA '24. IEEE Press, 2025, p. 1080–1096. [Online]. Available: <https://doi.org/10.1109/ISCA59077.2024.00082>
- [79] J. Zhang and A. Jog, “Tlp-aware cooperative scheduling for efficient gpu memory system utilization,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. ACM, 2017, pp. 93–104.
- [80] H. Zhu, A. Phanishayee, and G. Pekhimenko, “Daydream: Accurately estimating the efficacy of optimizations for dnn training,” in *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2020, pp. 337–352. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/zhu-hongyu>